

Advanced CI/CD & Deployment Strategies

Implementation Steps

- **Jenkins setup:** Install Jenkins locally or use a cloud VM. Install Git, Docker, and the Jenkins plugins Pipeline, Git, GitHub Integration, Credentials Binding, and Docker Pipeline. Add the Jenkins user to the Docker group, then restart Jenkins if required.
- **GitHub connection:** Create a repository containing the application, Dockerfile, Jenkinsfile, and a basic test. In Jenkins, create a Pipeline job, select “Pipeline script from SCM,” choose Git, enter the repository URL, select the branch, and set the script path to Jenkinsfile.
- **Credentials:** Add a GitHub token if the repository is private. Add Docker Hub username/password or a registry token as a Jenkins username/password credential with ID dockerhub-creds. Never hard-code secrets in the Jenkinsfile.
- **Webhook:** In GitHub repository Settings @ Webhooks, add the Jenkins webhook endpoint. In the Jenkins job, enable the GitHub hook trigger. For a local Jenkins server, expose it through a secure tunnel or use polling instead.

Pipeline Stages

- Checkout retrieves the selected Git revision from GitHub.
- Build installs dependencies or compiles the application.
- Test runs an automated health check or unit test and fails the build when the test fails.
- Package builds a Docker image using a unique build tag.
- Push authenticates to the registry and publishes the image.

Environment Variables

The pipeline uses DOCKER_IMAGE and IMAGE_TAG environment variables. The Jenkins BUILD_NUMBER variable is used to create a unique Docker image tag for each pipeline execution.

Registry credentials are accessed using Jenkins credentials binding. Passwords and access tokens are not stored directly in the Jenkinsfile.

Blue-Green Deployment

Blue-Green deployment uses two production environments. The Blue environment is the currently active version, while the Green environment receives the new release.

The new version is deployed and tested in Green. After successful validation, user traffic is switched from Blue to Green. If the new release fails, traffic can be switched back to Blue for rapid rollback.

The main benefits are low downtime and fast rollback. The main limitation is the additional infrastructure required to maintain two environments.

Rolling Deployment

Rolling deployment updates application instances gradually. Instead of replacing every instance at once, a small group of instances is updated while the remaining instances continue serving requests.

Rolling deployment reduces infrastructure requirements and can maintain availability. However, multiple application versions may run at the same time, and rollback may take longer than with Blue-Green deployment.

Jenkinsfile

```
pipeline {
    agent any

    environment {
        DOCKER_IMAGE = "nishasubramaniyan/jenkins-docker-demo"
        IMAGE_TAG = "${BUILD_NUMBER}"
    }

    triggers {
        githubPush()
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Build') {
            steps {
                sh '''
                    python3 --version
                    python3 -m venv venv
                    . venv/bin/activate
                    pip install --upgrade pip
                    pip install -r requirements.txt
                '''
            }
        }

        stage('Test') {
            steps {
                sh '''
                    . venv/bin/activate
                    pytest -v
                '''
            }
        }

        stage('Package') {
            steps {
                sh '''
                    docker build \
                        -t ${DOCKER_IMAGE}:${IMAGE_TAG} \
                        -t ${DOCKER_IMAGE}:latest .
                '''
            }
        }
    }
}
```

```

    }
}

stage('Push') {
    steps {
        withCredentials([
            usernamePassword(
                credentialsId: 'dockerhub-creds',
                usernameVariable: 'DOCKER_USERNAME',
                passwordVariable: 'DOCKER_PASSWORD'
            )
        ]) {
            sh '''
                echo "$DOCKER_PASSWORD" | docker login \
                    --username "$DOCKER_USERNAME" \
                    --password-stdin

                docker push ${DOCKER_IMAGE}:${IMAGE_TAG}
                docker push ${DOCKER_IMAGE}:latest

                docker logout
            '''
        }
    }
}

post {
    always {
        sh 'rm -rf venv || true'
    }

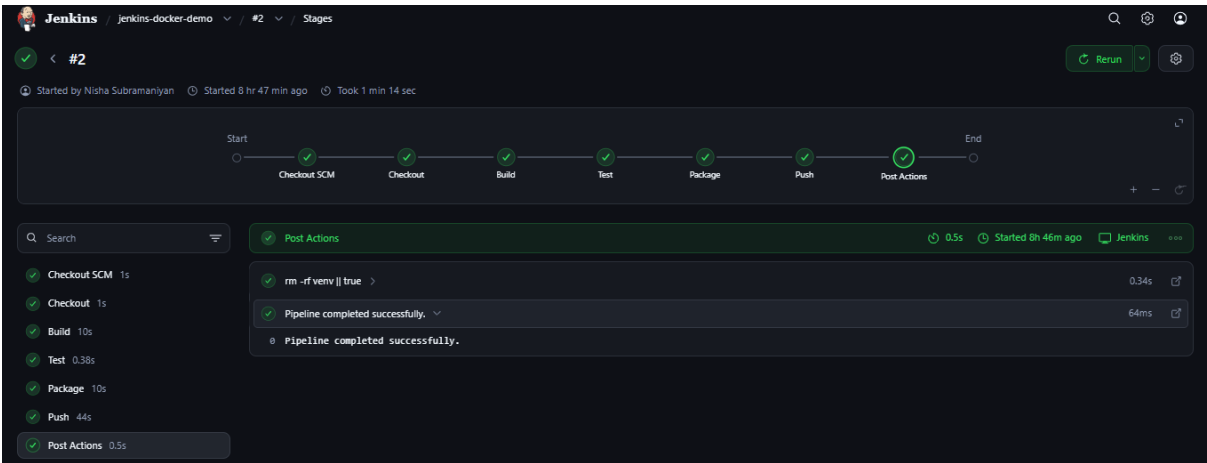
    success {
        echo 'Pipeline completed successfully.'
    }

    failure {
        echo 'Pipeline failed. Check the console output.'
    }
}
}

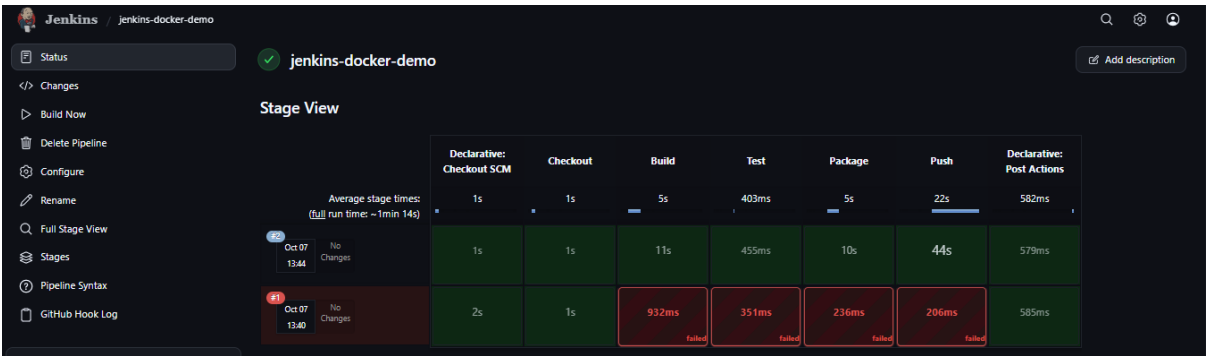
```

GitHub Repository Link: [jenkins-docker-demo.git](#)

Pipeline Screenshot



Successful Build Screenshot



Docker Image Screenshot

